Phần 1: Phân tích nguyên nhân & cơ chế

1. Tại sao Notepad bị "Not Responding"?

- **Cơ chế nạp toàn bộ vào RAM (Buffer full-load):** Trình biên tập văn bản giao diện đồ họa (GUI) như Notepad không được thiết kế cho tệp dữ liệu lớn (Big Data/Logs). Khi mở tệp, Notepad cố gắng đọc toàn bộ 800 MB dữ liệu, giải mã văn bản và phân bổ toàn bộ cấu trúc DOM/bộ nhớ đệm vào RAM cùng lúc.
- **Tắc nghẽn luồng xử lý UI (UI Thread Blocking):** Quá trình cấp phát bộ nhớ và render hàng triệu dòng text chiếm trọn luồng xử lý chính của ứng dụng. Hệ điều hành Windows không nhận được phản hồi (heartbeat) từ vòng lặp tin nhắn của cửa sổ Notepad, dẫn đến trạng thái đóng băng và kích hoạt thông báo `"Not Responding"`.

2. Tại sao lệnh cat bị lỗi trên CMD nhưng lại chạy được trên PowerShell?

- **Trên Windows CMD (Command Prompt):** `cat` là tiện ích xuất phát từ các hệ điều hành họ Unix/Linux (`concatenate`). CMD là trình thông dịch dòng lệnh cũ của Windows; nó không có tệp thực thi `cat.exe` và cũng không cài sẵn lệnh nội tại này (trên CMD, lệnh tương đương để đọc file là `type`). Do đó, CMD trả về lỗi:

Plaintext
```
'cat' is not recognized as an internal or external command, operable program or batch file.
```

- **Trên PowerShell:** PowerShell hỗ trợ cơ chế bí danh lệnh (**Alias**) nhằm giúp người dùng quen thuộc với Linux/Bash dễ dàng thao tác. Trong PowerShell:
    - `cat` thực chất là một alias trỏ trực tiếp đến cmdlet chuẩn: `Get-Content`.
    - Bạn có thể kiểm tra cơ chế này bằng lệnh: `Get-Alias cat` (kết quả trả về `ResolvedCommand: Get-Content`).

Phần 2: Bộ lệnh chuẩn trên PowerShell

1. Xem toàn bộ nội dung tệp theo luồng (Tránh tràn RAM)

Để đọc một tệp lớn 800 MB mà không làm ngốn cạn bộ nhớ, cần đọc và đẩy dữ liệu theo luồng (stream chunking) thay vì nạp toàn bộ một lần:

PowerShell

```
Get-Content -Path .\server_log_800MB.log -ReadCount 1000
```

Giải thích tối ưu: Tham số `-ReadCount 1000` yêu cầu PowerShell gửi từng khối 1.000 dòng qua pipeline mỗi lần, giúp giảm thiểu đáng kể chi phí cấp phát đối tượng (overhead) của PowerShell so với việc xử lý từng dòng đơn lẻ hoặc gom toàn bộ file vào RAM.

Nếu chỉ muốn xuất nhanh ra màn hình tương tự như CMD mà nhẹ tài nguyên nhất, bạn cũng có thể tận dụng:

PowerShell
```
Get-Content .\server_log_800MB.log
```

2. Chỉ xem 10 dòng cuối cùng (Kiểm tra lỗi mới nhất)

Không cần duyệt qua toàn bộ 800 MB dữ liệu từ đầu đến cuối; sử dụng tham số `-Tail` để đọc ngược từ cuối tệp:

PowerShell
```
Get-Content -Path .\server_log_800MB.log -Tail 10
```

- **Mẹo trực hệ thống lúc 12h đêm (Real-time Monitoring):** Nếu muốn theo dõi trực tiếp các dòng log mới phát sinh theo thời gian thực (tương tự `tail -f` trên Linux):

PowerShell
```
Get-Content -Path .\server_log_800MB.log -Tail 10 -Wait
```

(Nhấn `Ctrl + C` khi muốn dừng theo dõi).